



SPRING-BOOT 3.5 → 4.0 · MINI GUIDE



Hibernate & JPA: 3.5 → 4.0

Ten data-layer changes, mostly the quiet kind

One cut from the full Spring Break Cheat Sheets. Every card verified against Spring Boot 3.5.16 and 4.0.7. Real error output, real fixes, from the spring-break test suite. Steve Poole.

About this mini guide

This is one cut from **Spring Break Cheat Sheets**, the field guide to every breaking change between Spring Boot 3.5 and 4.0. Each card covers one change, verified by running real code on 3.5.16 and 4.0.7. One change, one card, one page: the error you'll see, the fix, and the traps.

HOW TO READ A CARD

TIER	How it fails. Tier 1 won't build, Tier 2 won't start, Tier 3 behaves differently. Tier 3 is the dangerous one: nothing goes red, your data just changes shape.
IMPACT	Effort per occurrence once found. S is find-and-replace, M needs an hour of care, L needs a plan.
OPENREWRITE	The badge means the Boot 4.0 recipe fixes this one for you. Run it before touching anything by hand.
WHAT YOU'LL SEE	The real output, so you can match it against your own build.
THE FIX	Before and after code, plus the steps when a rename isn't enough.
WATCH OUT	The traps that make this change look like a different bug.
VERIFY	The command that proves you're done.

THE SERIES

Spring Batch: 3.5 → 4.0 Eight changes between your jobs and 4.0

Hibernate & JPA: 3.5 → 4.0 Ten data-layer changes, mostly the quiet kind **THIS GUIDE**

Security & SSO: 3.5 → 4.0 Seven auth changes you cannot ignore

Data & Messaging: 3.5 → 4.0 Nine changes across your stores and streams

Testing: 3.5 → 4.0 Seven test-suite breaks, one clean build away

Observability & Ops: 3.5 → 4.0 Six changes your dashboards will notice first

Jackson & JSON: 3.5 → 4.0 Nine JSON changes everyone hits

Web & HTTP: 3.5 → 4.0 Eight changes at the HTTP layer

Core Platform & Build: 3.5 → 4.0 Twenty platform changes underneath everything

The Big Jobs: L-Impact Changes The six changes that eat your sprint

The full guide collects every change in one book, with a complete planning matrix and the OpenRewrite robot list. github.com/spoole167/spring-break



About HeroDevs

HeroDevs keeps critical open source software secure after the community stops patching it. Never-Ending Support (NES) covers CVEs in Spring Boot 3.5, Java, and dozens of other end-of-life frameworks, so your team migrates on its own schedule instead of the vendor's deadline.

This book exists because migrations fail from surprise more often than difficulty. Every card is a fact, not a guess: verified by running real code on Spring Boot 3.5.16 and 4.0.7.

The complete book, every subsystem in one volume, is coming to LeanPub. Watch github.com/spoole167/spring-break for the link.

herodevs.com

The Planning Matrix

Every change in this book, in book order. Impact is the effort per occurrence once found (S small, M moderate, L large), not how many you have. Auto means the OpenRewrite Boot 4.0 recipe handles it.

10 CHANGES

6 × TIER 1 - WON'T BUILD

3 × TIER 2 - WON'T START

1 × TIER 3 - BEHAVES DIFFERENTLY

TIER	CHANGE	IMPACT	AUTO
1	@EntityScan Relocated to persistence.autoconfigure	S	•
1	Hibernate EmptyInterceptor Removed	S	•
1	hibernate-jpamodelgen Renamed	S	Y
1	Hibernate SelectionQuery.setOrder() Removed	S	•
1	Session.delete() Removed	M	Y
1	Hibernate @Where and @OrderBy Removed	S	•
2	CascadeType.SAVE_UPDATE Removed in Hibernate 7	M	Y
2	Version-Specific Hibernate Dialects Removed	S	Y
2	Hibernate Untyped Join Query Rejected at Runtime	S	•
3	Hibernate Native Query Date Return Types Changed	M	•

The Robot List vs The Human List

The left column is what the OpenRewrite Boot 4.0 recipe fixes while you get coffee. The right column needs a person who understands your code. This split is the honest size of the migration: the renames are free, the behaviour changes are the work.

THE ROBOT LIST

Run the recipe, review the diff, done.

hibernate-jpamodelgen Renamed

Session.delete() Removed

CascadeType.SAVE_UPDATE Removed in Hibernate 7

Version-Specific Hibernate Dialects Removed

THE HUMAN LIST

Read the card. Check your code. Decide.

@EntityScan Relocated to persistence.autoconfigure

Hibernate EmptyInterceptor Removed

Hibernate SelectionQuery.setOrder() Removed

Hibernate @Where and @OrderBy Removed

Hibernate Untyped Join Query Rejected at Runtime

Hibernate Native Query Date Return Types Changed

TIER 1

Tier 1 - Won't Build

6 cards

1. @EntityScan Relocated to `persistence.autoconfigure`
2. Hibernate EmptyInterceptor Removed
3. hibernate-jpamodelgen Renamed
4. Hibernate SelectionQuery.setOrder() Removed
5. Session.delete() Removed
6. Hibernate @Where and @OrderBy Removed

@EntityScan Relocated to persistence.autoconfigure

Tier 1 - Won't Build

Hibernate & JPA

Impact: S

@EntityScan moved from `org.springframework.boot.autoconfigure.domain` to `org.springframework.boot.persistence.autoconfigure`. Old import doesn't compile.

WHAT CHANGED

@EntityScan moved from `org.springframework.boot.autoconfigure.domain` to `org.springframework.boot.persistence.autoconfigure`. The annotation's behaviour is unchanged; only the package differs.

WHY

Spring Boot 4.0 extracted persistence-related auto-configuration into a dedicated `spring-boot-persistence` module. This gives persistence concerns a cleaner home and makes the module boundaries explicit.

SCOPE CHECK

Search for `org.springframework.boot.autoconfigure.domain.EntityScan` across all Java/Kotlin sources. Typically appears on main application classes or dedicated JPA configuration classes.

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/main/java/com/example/EntityScanApp.java:[3,52]
  error: package org.springframework.boot.autoconfigure.domain does not exist
```

THE FIX

Import

BEFORE

```
import org.springframework.boot.autoconfigure.domain.EntityScan;
```

AFTER

```
import org.springframework.boot.persistence.autoconfigure.EntityScan;
```

HOW TO FIX

Update the import.

Replace `org.springframework.boot.autoconfigure.domain.EntityScan` with `org.springframework.boot.persistence.autoconfigure.EntityScan`. One import change per file; no other modifications needed.

WATCH OUT

Other classes from `org.springframework.boot.autoconfigure.domain` may have moved too. If you import anything else from that package, check the Spring Boot 4.0 Migration Guide for the new location.

FURTHER INFO

Several JPA autoconfiguration classes moved together; EntityScan is the most commonly used of the set.

Hibernate EmptyInterceptor Removed

Tier 1 - Won't Build

Hibernate & JPA

Impact: S

WHAT CHANGED

`org.hibernate.EmptyInterceptor` has been removed. The `org.hibernate.Interceptor` interface now has default (no-op) implementations for all its methods, so the abstract base class serves no purpose.

WHY

When Hibernate 6.0 added default methods to `Interceptor`, `EmptyInterceptor` became redundant. Hibernate 7.0 removed it as part of the overall API cleanup.

SCOPE CHECK

Search for `EmptyInterceptor` and `extends EmptyInterceptor` across all Java/Kotlin sources.

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/main/java/com/example/MyHibernateInterceptor.java:[5,43]
error: cannot find symbol
  symbol: class EmptyInterceptor
  location: package org.hibernate
```

THE FIX

Before — extending `EmptyInterceptor`

BEFORE

```
import org.hibernate.EmptyInterceptor;

public class MyInterceptor extends EmptyInterceptor {
    @Override
    public boolean onLoad(Object entity, Object id,
        Object[] state, String[] propertyNames, Type[] types) {
        // custom logic
        return false;
    }
}
```

After — implementing `Interceptor` directly

ADDED

```
import org.hibernate.Interceptor;

public class MyInterceptor implements Interceptor {
    @Override
    public boolean onLoad(Object entity, Object id,
        Object[] state, String[] propertyNames, Type[] types) {
        // custom logic
        return false;
    }
    // All other methods have default no-op implementations
}
```

HOW TO FIX

Implement Interceptor directly.

Replace `extends EmptyInterceptor` with `implements Interceptor` and remove the `EmptyInterceptor` import. Override only the methods you use.

WATCH OUT

Some method signatures on `Interceptor` changed in Hibernate 6.x (e.g., `Serializable id` became `Object id`). Verify your overridden signatures match exactly. A mismatch compiles as a new overload and Hibernate never calls it.

FURTHER INFO

`EmptyInterceptor` was a convenience class giving no-op implementations of all `Interceptor` methods. Deprecated in Hibernate 6.0, removed in 7.0.

spring-break module: `hibernate-empty-interceptor-removed` [Hibernate 7.0 Migration Guide](#)

hibernate-jpamodelgen Renamed

Tier 1 - Won't Build

Hibernate & JPA

Impact: S

OpenRewrite

WHAT CHANGED

Hibernate 7.0 (shipped with Spring Boot 4.0) renamed the annotation processor artifact from

`org.hibernate.orm:hibernate-jpamodelgen` to

`org.hibernate.orm:hibernate-processor`.

WHY

The processor now validates HQL/JPQL queries at compile time and generates type-safe query methods as well as the JPA static metamodel. The new name reflects the broader scope.

SCOPE CHECK

Search for `hibernate-jpamodelgen` in all POM and Gradle files. Also check

`annotationProcessorPaths` in `maven-compiler-plugin` configuration.

WHAT YOU'LL SEE

```
$ mvn compile
```

```
[WARNING] The POM for org.hibernate.orm:hibernate-jpamodelgen:jar:7.0.0 is missing, no dependency information available
```

```
[ERROR] Failed to execute goal on project my-service: Could not resolve dependencies:
```

```
Could not find artifact org.hibernate.orm:hibernate-jpamodelgen:jar:7.0.0
in central (https://repo.maven.apache.org/maven2)
```

THE FIX

pom.xml — annotation processor dependency

BEFORE

```
<dependency>
  <groupId>org.hibernate.orm</groupId>
  <artifactId>hibernate-jpamodelgen</artifactId>
  <scope>provided</scope>
</dependency>
```

AFTER

```
<dependency>
  <groupId>org.hibernate.orm</groupId>
  <artifactId>hibernate-processor</artifactId>
  <scope>provided</scope>
</dependency>
```

build.gradle — annotation processor

BEFORE

```
annotationProcessor 'org.hibernate.orm:hibernate-jpamodelgen'
```

AFTER

```
annotationProcessor 'org.hibernate.orm:hibernate-processor'
```

```
maven-compiler-plugin annotationProcessorPaths
```

BEFORE

```
<path>
  <groupId>org.hibernate.orm</groupId>
  <artifactId>hibernate-jpamodelgen</artifactId>
</path>
```

AFTER

```
<path>
  <groupId>org.hibernate.orm</groupId>
  <artifactId>hibernate-processor</artifactId>
</path>
```

HOW TO FIX

Rename the artifact.

Replace `hibernate-jpamodelgen` with `hibernate-processor` in your POM or Gradle build file. The Spring Boot BOM manages the version: change only the artifact ID.

Check compiler plugin configuration.

If you declared the processor in `maven-compiler-plugin`'s `annotationProcessorPaths`, update it there too.

WATCH OUT

The new `hibernate-processor` validates your HQL queries at compile time by default. Invalid queries that previously failed only at runtime now fail the build. That is a feature, but it can surface hidden bugs.

If your IDE configures annotation processors separately (e.g., IntelliJ's settings), update that too. Otherwise the metamodel generates in Maven but not in the IDE.

FURTHER INFO

Driven by Hibernate 7.0. See also: [cascade-save-update](#), [session-delete-removed](#).

[Spring-Break Demo](#) [Hibernate 7 migration guide](#)

Hibernate SelectionQuery.setOrder() Removed

Tier 1 - Won't Build

Hibernate & JPA

Impact: S

WHAT CHANGED

The `@Incubating` `setOrder(List<Order>)` method on `SelectionQuery` (and `Query`) was removed in Hibernate 7.0. It had been added in Hibernate 6.x as part of Jakarta Data integration work but was never promoted to stable API.

WHY

The method was always experimental. Hibernate 7.0 completed the Jakarta Data integration under a more stable API surface and removed the incubating intermediate.

SCOPE CHECK

Search for `.setOrder()` on query objects across all repository and data-access classes. Rare outside code written to try the Hibernate 6.x incubating API.

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/main/java/com/example/HibernateQueryUsage.java:[14,15]
error: cannot find symbol
  symbol:   method setOrder(java.util.List)
  location: interface SelectionQuery<Product>
```

THE FIX

Programmatic order via removed `setOrder()`

BEFORE

```
SelectionQuery<Product> query =
    session.createQuery("from Product", Product.class);
query.setOrder(List.of(Order.asc(Product.class, "name")));
```

Fix: inline ORDER BY in HQL

ADDED

```
SelectionQuery<Product> query =
    session.createQuery("from Product order by name asc", Product.class);
```

HOW TO FIX

Move ordering into the HQL query string.

Add an `ORDER BY` clause directly to the HQL/JPQL string. This is the most portable approach and works on both 3.5 and 4.0.

Use CriteriaQuery with Order.

For fully programmatic ordering, use the JPA Criteria API:

```
criteriaQuery.orderBy(builder.asc(root.get("name"))).
```

WATCH OUT

`org.hibernate.query.Order` (the parameter type) is also affected. If you imported it, that import can be removed once you stop using `setOrder`.

FURTHER INFO

`setOrder()` only existed during the Hibernate 6.x window. Code that adopted it must move to HQL ORDER BY or the SelectionSpecification API.

spring-break module: `hibernate-query-setorder-removed` [Hibernate 7.0 Migration Guide](#)

Session.delete() Removed

Tier 1 - Won't Build

Hibernate & JPA

Impact: M

🔄 OpenRewrite

WHAT CHANGED

Hibernate 7.0 removed the legacy `Session.delete()` method. The JPA-standard `EntityManager.remove()` (which maps to `Session.remove()`) is the only way to delete entities. `Session.save()`, `Session.update()`, and `Session.saveOrUpdate()` were also removed.

WHY

These methods were Hibernate-proprietary duplicates of the JPA standard API. Having two ways to do the same thing caused confusion about cascade behaviour and flush timing. The JPA methods have clearer semantics.

SCOPE CHECK

Search for `session.delete()`, `session.save()`, `session.update()`, and `session.saveOrUpdate()`. Also search for `Session.delete` in any utility or base repository classes.

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/main/java/com/example/repo/CustomRepoImpl.java:[28,16]
error: cannot find symbol
  symbol:   method delete(com.example.model.Order)
  location: variable session of type org.hibernate.Session
```

THE FIX

Deleting an entity

BEFORE

```
session.delete(order);
```

AFTER

```
session.remove(order);
```

Saving an entity

BEFORE

```
session.save(newOrder);
```

AFTER

```
session.persist(newOrder);
```

Updating an entity

BEFORE

```
session.update(existingOrder);
```

AFTER

```
session.merge(existingOrder);
```

Save-or-update pattern

BEFORE

```
session.saveOrUpdate(order);
```

AFTER

```
session.merge(order);
```

HOW TO FIX

Replace with JPA standard methods.

`delete()` becomes `remove()`, `save()` becomes `persist()`, `update()` becomes `merge()`, `saveOrUpdate()` becomes `merge()`.

Prefer EntityManager over Session.

If you're unwrapping the `Session` just to call these methods, consider using `EntityManager` directly. It has the same `persist()`, `merge()`, and `remove()` methods.

WATCH OUT

`merge()` returns the managed instance; `remove()` requires a managed entity. If you called `saveOrUpdate()` did not. If your code used the original `delete()` on detached entities, `merge()` or re-fetch them first. Otherwise you get an `IllegalArgumentException` at runtime.

FURTHER INFO

Driven by Hibernate 7.0, upstream of Spring Boot 4.0. The methods were deprecated in Hibernate 6.0. See also: [cascade-save-update](#), [hibernate-dialect-removal](#).

spring-break module: [hibernate-session-delete](#) [Hibernate 7 migration guide](#)

Hibernate @Where and @OrderBy Removed

Tier 1 - Won't Build

Hibernate & JPA

Impact: S

Hibernate's @Where and @OrderBy are removed. Replace with @SQLRestriction and @SQLOrder.

WHAT CHANGED

`@org.hibernate.annotations.Where` and `@org.hibernate.annotations.OrderBy` have been removed. The replacements are `@org.hibernate.annotations.SQLRestriction` and `@org.hibernate.annotations.SQLOrder`, which take the same SQL fragment arguments.

WHY

The new annotation names make it explicit that the argument is a raw SQL fragment, not a JPQL or HQL expression. This avoids confusion with the JPA `@OrderBy` (which takes attribute names) and with Hibernate's HQL-level features.

SCOPE CHECK

Search for `@Where` and `@OrderBy` from the `org.hibernate.annotations` package across all entity classes and embeddables. Also check `@WhereJoinTable` if used.

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/main/java/com/example/User.java:[5,39]
error: cannot find symbol
  symbol: class Where
  location: package org.hibernate.annotations
[ERROR] /src/main/java/com/example/User.java:[6,39]
error: cannot find symbol
  symbol: class OrderBy
  location: package org.hibernate.annotations
```

THE FIX

Entity annotations

BEFORE

```
import org.hibernate.annotations.Where;
import org.hibernate.annotations.OrderBy;

@OneToMany
@Where(clause = "deleted = false")
@OrderBy(clause = "name desc")
private List<Post> posts;
```

AFTER

```
import org.hibernate.annotations.SQLRestriction;
import org.hibernate.annotations.SQLOrder;

@OneToMany
@SQLRestriction("deleted = false")
@SQLOrder("name desc")
private List<Post> posts;
```

HOW TO FIX

Replace @Where with @SQLRestriction.

Change the annotation name and attribute: `@Where(clause = "...")` becomes `@SQLRestriction("...")`. The SQL fragment is the same.

Replace @OrderBy with @SQLOrder.

Change the annotation name and attribute: `@OrderBy(clause = "...")` becomes `@SQLOrder("...")`.

WATCH OUT

Only Hibernate's `@OrderBy` needs replacing. JPA's `@jakarta.persistence.OrderBy` takes attribute names, not SQL, and is unaffected.

FURTHER INFO

`@SQLRestriction` and `@SQLOrder` arrived in Hibernate 6.3; the old annotations were deprecated then and removed in 7.0.

spring-break module: hibernate-where-orderby-removed [Hibernate 7.0 Migration Guide](#)

TIER 2

Tier 2 - Won't Start

3 cards

1. CascadeType.SAVE_UPDATE Removed in
Hibernate 7
2. Version-Specific Hibernate Dialects Removed
3. Hibernate Untyped Join Query Rejected at
Runtime

CascadeType.SAVE_UPDATE Removed in Hibernate 7

Tier 2 - Won't Start

Hibernate & JPA

Impact: M

OpenRewrite

Hibernate 7 removed the proprietary `CascadeType.SAVE_UPDATE`. Entities using this cascade type fail at startup or throw `MappingException` when the `SessionFactory` is built.

WHAT CHANGED

`org.hibernate.annotations.CascadeType.SAVE_UPDATE` was a Hibernate-specific extension to JPA cascade types. It triggered cascade behaviour on Hibernate's proprietary `Session.saveOrUpdate()` method. Hibernate 7 removed both `saveOrUpdate()` and the corresponding cascade type. Code using the Hibernate-specific `@Cascade` annotation with `SAVE_UPDATE` breaks at entity mapping time.

WHY

The `saveOrUpdate()` method was deprecated in Hibernate 6 as part of aligning with the JPA `EntityManager` API. Hibernate 7 completed the removal. The standard JPA `CascadeType.PERSIST` and `CascadeType.MERGE` cover the same use cases without vendor lock-in.

SCOPE CHECK

Search for `CascadeType.SAVE_UPDATE` and `@Cascade` (the Hibernate annotation, not JPA). Every hit is a mapping that will fail at startup. Also search for `session.saveOrUpdate()`; that method is gone too.

WHAT YOU'LL SEE

```
org.springframework.beans.factory.BeanCreationException:
    Error creating bean with name 'entityManagerFactory'
...
Caused by: org.hibernate.MappingException:
    Unrecognized legacy cascade style: save-update
    at org.hibernate.boot.model.internal.EntityBinder.bindEntityAnnotation(...)
    at org.hibernate.boot.model.internal.AnnotationBinder.bindClass(...)
...
APPLICATION FAILED TO START
---
Description:
Failed to initialize JPA EntityManagerFactory:
Unrecognized legacy cascade style: save-update
```

THE FIX

Using Hibernate `@Cascade` annotation

BEFORE

```
import org.hibernate.annotations.Cascade;
import org.hibernate.annotations.CascadeType;

@OneToMany(mappedBy = "order")
@Cascade(CascadeType.SAVE_UPDATE)
private List<OrderItem> items;
```

AFTER

```
@OneToMany(mappedBy = "order", cascade = {
    javax.persistence.CascadeType.PERSIST,
    javax.persistence.CascadeType.MERGE
})
private List<OrderItem> items;
```

Using JPA cascade with Hibernate extras

BEFORE

```
@OneToMany(mappedBy = "parent")
@Cascade({CascadeType.SAVE_UPDATE, CascadeType.DELETE})
private Set<Child> children;
```

AFTER

```
@OneToMany(mappedBy = "parent", cascade = {
    CascadeType.PERSIST, CascadeType.MERGE, CascadeType.REMOVE
})
private Set<Child> children;
```

HOW TO FIX

Replace with JPA standard cascades.

Swap `CascadeType.SAVE_UPDATE` for the combination of `CascadeType.PERSIST` and `CascadeType.MERGE`. These two JPA-standard cascades cover the same persist-or-update behaviour that `SAVE_UPDATE` provided.

Remove `@Cascade` if redundant.

If your entity already has `cascade = CascadeType.ALL` on the JPA annotation, the Hibernate `@Cascade` is redundant. Delete it entirely.

WATCH OUT

`CascadeType.PERSIST` alone does not cover the "update existing detached entity" case. You need both `PERSIST` and `MERGE` to match the old `SAVE_UPDATE` behaviour.

Some code generators and legacy mapping tools emit `SAVE_UPDATE` by default. Re-run generation after upgrading.

If you were using `Session.saveOrUpdate()` directly in DAO code, that method is also removed. Replace with `entityManager.merge()` or `entityManager.persist()`.

FURTHER INFO

Arrives via `spring-boot-starter-data-jpa`, which pulls in Hibernate 7.0. See also: `session-delete-removed`, `hibernate-dialect-removal`.

[spring-break module: hibernate-cascade-removal](#) [Hibernate 7 migration guide](#) [Spring Boot 4.0 Migration Guide](#)

Version-Specific Hibernate Dialects Removed

Tier 2 - Won't Start

Hibernate & JPA

Impact: S

🔄 OpenRewrite

WHAT CHANGED

Hibernate 7 removed all version-specific dialect classes

(`MySQL57Dialect` ,
`MySQL8Dialect` ,
`PostgreSQL95Dialect` ,
`PostgreSQL10Dialect` ,
`Oracle12cDialect` ,
`SQLServer2016Dialect` , etc.).

Only the base dialect classes remain (`MySQLDialect` ,
`PostgreSQLDialect` ,
`OracleDialect` ,
`SQLServerDialect`). Hibernate now auto-detects the database version from the JDBC connection and configures features accordingly.

WHY

A dialect class per database version created a combinatorial explosion. Runtime detection removes the maintenance burden and the risk of choosing the wrong class.

SCOPE CHECK

Search for `hibernate.dialect` in `application.properties` , `application.yml` , `persistence.xml` , and any programmatic `LocalContainerEntityManagerFactoryBean` configuration. Also check test configurations and Docker Compose profiles.

WHAT YOU'LL SEE

```
org.springframework.beans.factory.BeanCreationException:
  Error creating bean with name 'entityManagerFactory'
...
Caused by: java.lang.ClassNotFoundException:
  org.hibernate.dialect.MySQL57Dialect
  at java.base/jdk.internal.loader.BuiltinClassLoader.loadClass(BuiltinClassLoader.java:641)
  at java.base/jdk.internal.loader.ClassLoaders$AppClassLoader.loadClass(ClassLoaders.java:188)
  ...
APPLICATION FAILED TO START
---
Description:
Failed to configure a DataSource: Hibernate dialect
'org.hibernate.dialect.MySQL57Dialect' could not be found.
```

THE FIX

`application.properties`

BEFORE

```
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL57Dialect
```

AFTER

```
# Remove explicit dialect – Hibernate auto-detects from JDBC connection
```

application.properties — if you must be explicit

BEFORE

```
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQL95Dialect
```

AFTER

```
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect
```

persistence.xml

BEFORE

```
<property name="hibernate.dialect" value="org.hibernate.dialect.Oracle12cDialect"/>
```

AFTER

```
<property name="hibernate.dialect" value="org.hibernate.dialect.OracleDialect"/>
```

HOW TO FIX

Remove explicit dialect (recommended).

Delete the `spring.jpa.properties.hibernate.dialect` property entirely. Hibernate 7 detects the correct dialect from the JDBC connection metadata.

Switch to base dialect class.

If you need an explicit dialect for testing or CI (e.g., H2 in-memory), use the unversioned base class:

`MySQLDialect`, `PostgreSQLDialect`, `OracleDialect`, or `SQLServerDialect`.

VERIFY

App starts with no `DialectResolutionInfoMissingException`

WATCH OUT

If you use Testcontainers or H2 for tests, those test configs often set an explicit dialect. Check `src/test/resources` too.

The auto-detection requires an active JDBC connection at startup. If your application uses lazy datasource initialisation, you may still need an explicit base dialect class.

Some connection pools and frameworks set the dialect programmatically. Search Java code for `setProperty("hibernate.dialect" ...)` too.

FURTHER INFO

Driven by Hibernate 7.0, upstream of Spring Boot 4.0. The classes were deprecated in Hibernate 6.0. Affects spring-boot-starter-data-jpa consumers. See also: `hibernate-date-types`, `cascade-save-update`.

Hibernate Untyped Join Query Rejected at Runtime

Tier 2 - Won't Start

Hibernate & JPA

Impact: S

Hibernate 7 rejects untyped join queries with no explicit SELECT clause. A query that returned `Object[]` on Boot 3.5 throws `SemanticException` at runtime on Boot 4.0.

WHAT CHANGED

Hibernate 7.0 tightened validation of HQL queries. A query like `from Product p join p.category c` has two implicit query roots (Product and the joined Category). Without an explicit `SELECT` clause or result type, the return type is ambiguous. Hibernate 6.x resolved this as `Object[]`; Hibernate 7.0 rejects it at parse time with a `SemanticException`.

WHY

Silent return of `Object[]` hides intent and causes surprising `ClassCastException` at the point of use rather than at the query. A hard error forces explicit, clearer queries.

SCOPE CHECK

Search for `createQuery()` calls. JPQL queries via `EntityManager.createQuery(String, Class)` are also affected if they omit the `SELECT` clause with joins.

WHAT YOU'LL SEE

```
// Query without explicit result type
session.createQuery("from Product p join p.category c").getResultList();

// Boot 3.5 (Hibernate 6.x): runs fine, returns List<Object[]>

// Boot 4.0 (Hibernate 7.x): throws at runtime
org.hibernate.query.SemanticException:
  Query has no explicit SELECT and multiple query roots are defined.
  An explicit SELECT clause is required when multiple query roots exist
  or when the query contains a JOIN.
```

THE FIX

Add an explicit SELECT and result type

BEFORE

```
session.createQuery("from Product p join p.category c")
    .getResultList();
```

AFTER

```
session.createQuery(
    "SELECT p FROM Product p JOIN p.category c",
    Product.class
).getResultList();
```

Or use a typed tuple query

BEFORE

```
session.createQuery("from Product p join p.category c")
    .getResultList();
```

AFTER

```
session.createQuery(
    "SELECT p, c FROM Product p JOIN p.category c",
    Object[].class
).getResultList();
```

HOW TO FIX

Add explicit SELECT clause and result type (recommended).

All `Session.createQuery(String)` calls with join queries must include an explicit `SELECT` clause. Pass the expected result class as the second argument: `createQuery(hql, Entity.class)`.

Search for bare 'from' queries.

Grep for `createQuery("from` and `createQuery('from` in your codebase. Any query that starts with `from` and contains a join is at risk.

VERIFY

HQL join queries without explicit SELECT or result type execute without `SemanticException`

WATCH OUT

The query compiles and runs on Boot 3.5 without any warning. The failure only surfaces on Boot 4.0 at the first call site, not at startup. Tests that exercise query paths are the only way to catch this before production.

Simple queries without joins are unaffected: `from Product p` with a single root remains valid. Only multi-root or joined queries trigger the error.

FURTHER INFO

Introduced in Hibernate 7.0, upstream of Spring Boot 4.0. Affects any application using `Session.createQuery(String)` with join queries and no result type.

spring-break module: [hibernate-query-type-required](#) [Hibernate 7.0 Migration Guide](#) [Spring Boot 4.0 Migration Guide](#)

TIER 3

Tier 3 - Behaves Differently

1 card

1. Hibernate Native Query Date Return Types
Changed

Hibernate Native Query Date Return Types Changed

Tier 3 - Behaves Differently

Hibernate & JPA

Impact: M

Native SQL queries now return `java.time.LocalDate` instead of `java.sql.Date`. Code casting to the old types throws `ClassCastException` or silently loses time components.

WHAT CHANGED

Hibernate 7 (used by Spring Boot 4.0) changed the default Java types returned for SQL `DATE`, `TIME`, and `TIMESTAMP` columns in native queries. `java.sql.Date` became `java.time.LocalDate`, `java.sql.Time` became `java.time.LocalTime`, and `java.sql.Timestamp` became `java.time.LocalDateTime`.

WHY

The `java.sql` date/time classes are legacy wrappers around `java.util.Date` with known timezone and truncation bugs. Hibernate aligned its defaults with `java.time`: immutable, thread-safe, the modern standard.

SCOPE CHECK

Search for `nativeQuery = true`, `createNativeQuery`, and any cast to `java.sql.Date`, `java.sql.Time`, or `java.sql.Timestamp` in query result handling code.

WHAT YOU'LL SEE

```
// Native query – same code, different result type
Object result = em.createNativeQuery("SELECT created_date FROM orders WHERE id = 1")
    .getSingleResult();

// Before (Spring Boot 3.5)
result.getClass() → java.sql.Date

// After (Spring Boot 4.0)
result.getClass() → java.time.LocalDate

// ClassCastException at runtime
java.lang.ClassCastException:
  java.time.LocalDate cannot be cast to java.sql.Date

// Or subtler: time component silently lost
java.sql.Timestamp ts = (java.sql.Timestamp) result; // was Timestamp
// Now: java.time.LocalDateTime – no getTime() method
```

THE FIX

Fix native query result handling

BEFORE

```
java.sql.Date date = (java.sql.Date) row[2];
long millis = date.getTime();
```

AFTER

```
LocalDate date = (LocalDate) row[2];
long millis = date.atStartOfDay(ZoneOffset.UTC)
    .toInstant().toEpochMilli();
```

Typed native query

BEFORE

```
@Query(value = "SELECT created_date FROM orders", nativeQuery = true)
List<java.sql.Date> findAllDates();
```

AFTER

```
@Query(value = "SELECT created_date FROM orders", nativeQuery = true)
List<java.time.LocalDate> findAllDates();
```

Timestamp results

BEFORE

```
java.sql.Timestamp ts = (java.sql.Timestamp) row[3];
```

AFTER

```
LocalDateTime ts = (LocalDateTime) row[3];
```

HOW TO FIX

Update result type casts to java.time (recommended).

Replace all `java.sql.Date` casts with `java.time.LocalDate`, `java.sql.Time` with `java.time.LocalTime`, and `java.sql.Timestamp` with `java.time.LocalDateTime`.

Force old types with explicit `addScalar()`.

For legacy code that can't be updated, use Hibernate's `addScalar("col", StandardBasicTypes.DATE)` to force the old `java.sql` return types on specific queries.

VERIFY

SELECT queries return correct date/time types (check `ResultSet`)

WATCH OUT

This only affects native queries that return raw `Object[]` rows. JPQL queries with entity mappings or `@ColumnResult` type mappings are unaffected because the type is explicit.

`java.time.LocalDate` has no time component. If your code relied on `java.sql.Date.getTime()` returning midnight epoch millis, the replacement requires an explicit timezone conversion.

FURTHER INFO

Driven by Hibernate 7.0, upstream of Spring Boot 4.0, and documented in its migration guide. See also: [hibernate-dialect-removal](#).



Not ready to migrate?

Spring Boot 3.5 stops receiving fixes when open-source support ends. You have three sane options. Pick one deliberately.

1. **Pin 3.5 and track CVEs.** Free, but the clock is running and every new CVE is your team's problem to triage.
2. **Migrate incrementally with OpenRewrite.** The Boot 4.0 recipe automates the renames in this book; the behavioural changes still need eyes.
3. **HeroDevs NES for Spring.** Security patches for 3.5 beyond EOL, buying your team time to migrate on your schedule, not the calendar's.

Spring Boot 4.0 Migration Guide • github.com/spring-projects/spring-boot/wiki
OpenRewrite Boot 4.0 recipe • docs.openrewrite.org/recipes/java/spring/boot4
HeroDevs NES for Spring • herodevs.com/support/spring-nes
The test suite behind this book • github.com/spoole167/spring-break